

Structure de données et Mémoire : Les Pointeurs en C

Cours d'informatique - CEGEP

26 août 2026

1 Utilisation et visualisation

Un **pointeur** est une variable dont la valeur est l'adresse mémoire d'une autre variable. Contrairement aux langages de haut niveau (Python, Java), le C vous donne un accès direct à la quincaillerie de la machine.

- **L'opérateur d'adresse (&)** : Permet de récupérer l'adresse mémoire d'une variable.
- **L'opérateur de déréférencement (*)** : Permet d'accéder ou de modifier la valeur stockée à l'adresse pointée.

Visualisation de la mémoire : Supposons un entier $x = 42$ stocké à l'adresse `0x7ffee4`, et un pointeur p stockant cette adresse à l'adresse `0x7ffee0` :

Adresse	Nom de la variable	Valeur contenue
<code>0x7ffee0</code>	p (pointeur)	<code>0x7ffee4</code> ($\rightarrow$ pointe vers x)
...	...	...
<code>0x7ffee4</code>	x (entier)	42

2 Opérations courantes

Les manipulations fondamentales à maîtriser avec les pointeurs :

- `int *p = &x;` : Déclaration d'un pointeur et initialisation avec l'adresse de x .
- `*p = 100;` : Modification indirecte de la valeur de x via le pointeur (devient 100).
- `int *ptr = malloc(sizeof(int));` : Allocation dynamique de mémoire sur le *Heap* (tas).
- `free(ptr);` : Libération obligatoire de la mémoire allouée pour éviter les fuites (*memory leaks*).
- Passage par référence : Modifier une variable hors de sa portée initiale en passant son adresse à une fonction (ex : `void maFonction(int *var)`).

3 Complexité et Pièges critiques

Opération	Complexité	Attention / Piège
Accès / Modification directe	$O(1)$	Déréférencer un pointeur NULL plante le programme (SegFault).
Allocation dynamique (<code>malloc</code>)	$O(1)$ à $O(n)$	Oublier de libérer $\rightarrow$ Memory Leak .
Pointeur pendant (<i>Dangling pointer</i>)	—	Utiliser un pointeur dont la mémoire a déjà été libérée.

4 Applications

- **Structures de données dynamiques** : Impossible de coder des listes chaînées, des arbres binaires ou des graphes sans pointeurs pour lier les nœuds entre eux en mémoire.
- **Développement de noyaux et Systèmes d'exploitation** : Linux, Windows et macOS sont écrits en C/C++ et utilisent des pointeurs pour manipuler directement les registres matériels et la mémoire RAM.
- **Moteurs de jeux vidéo (Unreal Engine / C++)** : Gestion ultra-précise de la mémoire pour garantir un rendu fluide à 120 FPS sans que le *Garbage Collector* ne vienne geler le jeu.

- **Protocoles réseau et pilotes (Drivers) :** Manipulation directe des paquets réseaux reçus sur les cartes d'interface par le biais de tampons (*buffers*) mémoires.

5 Lore (Historique)

Les pointeurs sont indissociables de la création du langage C, développé par **Dennis Ritchie** entre 1972 et 1973 aux laboratoires Bell (AT&T). L'objectif était de concevoir un langage assez proche du matériel pour réécrire le système d'exploitation Unix tout en restant portable. C'est de là qu'émerge la célèbre citation de l'inventeur des tableaux de bord modernes : « *Le C vous permet de vous tirer une balle dans le pied aussi facilement que vous le souhaitez ; et le pointeur est le mécanisme qui permet d'y parvenir directement.* »